

Kart Racing Simulator

Technical Documentation and User Manual

Interactive Graphics Course Project
Three.js, WebGL, Vite and JavaScript ES Modules

Project summary. Kart Racing Simulator is a browser-based 3D arcade racing game developed with Three.js. The application lets the player choose a vehicle, select one of three themed circuits, configure a body color, and race either against an AI opponent or in time trial mode. The project combines imported vehicle assets, a team-authored procedural kart, spline-based track generation, decorative low-poly environments, real-time lighting, procedural textures, collision handling, HUD systems, minimap rendering, audio feedback and JavaScript-authored animations.

1. Project Overview

The goal of the project is to build a complete real-time 3D racing simulator for the browser while satisfying the Interactive Graphics requirements: hierarchical models, lights, texture variety, meaningful user interaction and animations implemented directly in JavaScript. The selected theme is an arcade racing experience inspired by compact, readable low-poly racing games.

The simulator contains three playable vehicles: Porsche Cayman GT4, Nissan Silvia S14 Kouki and a procedural kart. The Porsche and Silvia are external static models adapted for runtime use, while the kart is built in the project source code from Three.js primitives and is the main hierarchical model. The game also contains three tracks: Vegas Neon, Tropical Beach and Monaco Formula 1. Each track has its own geometry, palette, lighting configuration, props and gameplay tuning.

Main goals

- Create a coherent browser game rather than a static scene, with menu flow, race states, checkpoints, lap timing, collision response and finish feedback.
- Demonstrate hierarchical modeling through the procedural kart, including nested wheel pivots, steering pivots, suspension motion, a driver hierarchy, headlights and a moving steering wheel.
- Use real-time graphics techniques such as scene graph transforms, spline geometry, mesh materials, generated textures, custom shaders, fog, shadows and dynamic lights.
- Implement all animations in JavaScript: no imported animation clips are used from external models.

Requirement coverage summary

Requirement	Project implementation
Hierarchical models	Procedural kart with chassis group, wheel roll groups, front steering pivots, driver body hierarchy, arm pivots, steering wheel and headlights.
Lights and textures	Ambient, directional, spot and point lights; shadows; emissive materials; imported model textures; generated CanvasTexture maps; roughness and normal-style procedural maps; custom GLSL boost-pad shader.
User interaction	Setup menu, vehicle/track/mode/color selection, driving controls, handbrake, headlights toggle, camera modes, pause, restart, debug toggles and audio settings.
Animations	Wheel rolling, steering, kart suspension, driver movement, body roll, AI vehicle motion, birds, floating/spinning props, countdown lights, boost shader pulse, camera transitions and barrier bounce.

2. Environment, Libraries and External Assets

The project is a static web application. It runs in a modern desktop browser and renders the 3D scene through WebGL. Vite is used for local development and production builds, while the runtime is written as JavaScript ES modules.

Runtime environment

- **HTML and CSS:** page shell, responsive setup menu, HUD, pause menu, finish screen and runtime overlays.
- **JavaScript modules:** application state, scene setup, vehicle classes, track generation, input, physics, race systems, audio, camera, minimap and UI integration.
- **Three.js:** renderer, scene graph, cameras, lights, geometry, materials, textures, vectors, groups and loaders.
- **Vite:** development server, bundling and asset pipeline.

External libraries and tools

Library or tool	Use in the project
Three.js	Main rendering and scene graph library. It provides WebGLRenderer, PerspectiveCamera, lights, mesh primitives, BufferGeometry, materials, loaders and math utilities.
Vite	Local development server and production build system for the browser application.
@tweenjs/tween.js	Used for smooth runtime transitions such as the arcade barrier bounce response.
Web Audio API	Used by the audio manager for procedural engine audio and event feedback, with additional ambience samples where appropriate.

External models and assets

The Porsche Cayman GT4 model is imported from Sketchfab and licensed under CC-BY-NC-SA-4.0. The Nissan Silvia S14 Kouki model is also imported from Sketchfab, authored by Ddiaz Design, and licensed under CC-BY-NC-SA-4.0. Both models are used as static visual assets; their runtime motion is authored in JavaScript by the team. The procedural kart, tracks, most props, lights, particle effects, shader effects and generated textures are team-authored.

The project uses several audio files for crowd ambience, beach ambience and collision feedback. The runtime engine sound itself is procedural and generated with the Web Audio API. The imported model licenses are listed in this section; audio source and license details should be verified and recorded in the final submission materials before publication.

3. Application Architecture

The codebase is organized by responsibility. The entry point initializes the setup menu and lazily starts the Three.js scene only after the player chooses a configuration. Runtime logic is divided into systems so that physics, input, race state, camera, minimap and audio do not live inside a single monolithic file.

Area	Responsibility
src/main.js	Application entry point, setup menu mounting and transition into the 3D scene.
src/scene/	Renderer, scene, camera, lights, frame loop and runtime orchestration.
src/vehicles/	Vehicle interface, procedural kart, imported vehicle wrappers, loaders, wheel animation, headlights and color handling.
src/tracks/	Track data, spline generation, materials, barriers, checkpoints, boost pads and themed decorative props.
src/systems/	Input, arcade vehicle physics, track interaction, race manager, AI controller, camera controller, minimap, audio, ghost lap and wrong-way detection.
src/materials/	Procedural texture generation and custom shader material helpers.

Frame update loop

The frame loop computes a capped delta time, updates runtime tweens, consumes one-shot input actions, advances race and track systems, applies collision and boost state to the vehicle controller, updates vehicle transforms, animates wheels and props, updates audio, minimap and HUD state, then renders the scene. This order is important because the visual vehicle is updated only after the final physics state for the frame is known.

4. Tracks, Race Systems and Gameplay

All tracks use a shared spline-based generator. Track definitions store control points, road width, checkpoint progress values, boost pad positions, lighting theme and palette information. The generator samples the curve to create the road mesh, side barriers, centerline, checkpoints, boost pads and minimap bounds. Decorative props are added by themed modules so gameplay data remains separate from visual details.

Track themes

- **Vegas Neon:** a night circuit with neon lighting, casino-inspired signs, glowing skyline elements, decorative point lights and animated props such as floating dice and a Ferris wheel.
- **Tropical Beach:** a bright coastal track with sand, ocean, vegetation, huts, beach props, animated low-poly birds and lighter barriers.
- **Monaco Formula 1:** a narrow city circuit with marina props, yachts, buildings, flags, lamps, tight barriers and grandstands.

Race modes

The game supports two modes. Race mode creates an AI opponent using the same selected vehicle type as the player, runs a countdown, tracks laps and shows a simple position. Time Trial mode removes the AI opponent and focuses on lap timing and local best-lap records. The race manager owns state such as phase, countdown, current lap, checkpoint progress, total time, lap times, best lap and finish state.

Physics and interaction

The vehicle controller implements arcade driving with acceleration, braking, reverse, steering, handbrake, friction and speed limits. The track interaction system computes off-road grip, boost pad activation, barrier collision correction and simplified player-AI collision. Barriers are represented by oriented 2D collider boxes along the road edges, which keeps collision handling fast and stable for a browser project.

HUD, minimap and feedback

The runtime HUD shows speed, lap, time, checkpoint state, race position and performance information. A canvas minimap uses the track centerline and minimap bounds to draw the circuit, player marker, AI marker and checkpoint markers. Wrong-way detection compares the vehicle heading with the closest centerline direction and shows a dedicated warning overlay when needed.

5. Hierarchical Modeling

The main authored hierarchical model is the procedural kart. It is built entirely from Three.js primitives and nested groups. The hierarchy is designed so each animation affects the correct local structure: wheel roll groups spin around their own local axes, front steering pivots rotate only the front wheels, the driver head and arms respond to steering, and the chassis group can bounce or roll without manually updating every mesh.

Procedural kart hierarchy

The kart structure can be summarized as follows:

KartVehicle

group

KartChassis

body, frame, nose, side pods, seat, engine, bumpers

front axle / rear axle

front steering pivots

wheel roll pivots

wheel, hub, cap

rear wheel mounts

wheel roll pivots

wheel, hub, cap

KartDriver

head pivot

helmet, visor, stripe

shoulder pivots

upper arm

elbow pivot

forearm, hand

KartSteeringWheel

headlights, beams and targets

This structure is not just organizational. The driver animation exploits the hierarchy by rotating the head pivot, shoulders, elbows and hands according to steering and speed. The steering wheel rotates with player input, while the wheel pivots and suspension motion make the kart react visually to movement. This directly satisfies the hierarchical model animation requirement.

Imported vehicles

The Porsche and Silvia models are imported as static meshes, but their gameplay animation is authored in JavaScript. During setup, the code fits each model to a target length, clones materials per instance, collects wheel nodes and wraps them in new steering and roll pivot groups. Wheel rotation, steering, body lean and headlights are then driven by runtime vehicle state rather than imported animation clips.

6. Lights, Materials and Textures

The project uses a mix of real lights, emissive materials and generated textures. The base scene includes ambient light and a directional sun/moon light with shadows. Track themes override fog, sky gradients, light colors and shadow bounds to make each environment visually distinct.

Lighting

- **Global lighting:** ambient and directional lights create a readable base scene and shadowed geometry.
- **Vehicle lights:** Kart, Porsche and Silvia have toggleable headlights implemented with spot lights, emissive materials and translucent beam meshes.
- **Track lights:** Vegas uses neon emissive surfaces and decorative point lights; Monaco includes lamps and gantry lights; boost pads and checkpoints use emissive materials.
- **Feedback lights:** start gantry countdown lamps change from red to green according to race state.

Texture types

The project includes different texture categories. Imported vehicles use image textures such as color maps, normal maps and detail maps. Track surfaces use JavaScript-generated CanvasTexture maps for color, roughness and normal-style detail. UI and signage use canvas-generated text textures. The boost pad adds a custom ShaderMaterial with animated uniforms, producing a radial pulse effect in the fragment shader.

Low-poly material direction

The tracks and props intentionally use a low-poly arcade style. Materials are usually MeshStandardMaterial with flat shading, controlled roughness, limited metalness and strong color palettes. This gives the project readable geometry and keeps performance acceptable while still showing distinct themes.

7. Animations Implemented in JavaScript

The course requirements explicitly forbid importing animations. The project respects this constraint: no AnimationMixer, clipAction or imported animation clips are used. All movement is implemented through JavaScript updates, runtime state, trigonometric functions, shader uniforms or Tween.js interpolation.

Animated element	Implementation
Vehicle wheels and steering	Wheel rotation is computed from distance traveled; front steering pivots rotate from the controller steering value.
Procedural kart driver	Head, shoulders, elbows, hands and steering wheel rotate according to steering, speed and vibration timers.
Kart body and suspension	The chassis and wheel pivots bounce and roll using wheel rotation, speed ratio and steering load.
AI opponent	The AI controller samples the track centerline and moves a visible vehicle along the route with speed and curve tuning.
Boost pads	A GLSL shader receives an increasing <code>uTime</code> uniform to animate radial rings and spokes.
Environmental props	Props with <code>spin</code> , <code>float</code> or <code>flight</code> <code>userData</code> are animated in the main scene loop.
Barrier bounce	<code>Tween.js</code> interpolates a short rebound after significant barrier impact.

These animations are deterministic enough for gameplay but expressive enough for a live demonstration. They are also tied to scene graph structure: rotating a parent group moves all child components naturally, which is exactly the intended use of hierarchical modeling.

8. User Manual

When the application starts, the setup menu guides the player through track, vehicle and race mode selection. The player can then choose a body color before starting the countdown. After the countdown reaches GO, the vehicle becomes controllable and the HUD begins updating race state.

Controls

Action	Keys
Accelerate	W Or ArrowUp
Brake / reverse	S Or ArrowDown
Steer left	A Or ArrowLeft
Steer right	D Or ArrowRight
Handbrake	Space
Change camera	C
Toggle headlights	L
Restart race	R
Pause / menu	Esc

Camera and debug controls

The camera can be cycled during gameplay. The follow camera is the standard driving view, the top camera helps inspect the track, the hood camera gives a closer vehicle perspective, and the orbit/free camera supports exploration. Debug controls are available for development and demonstration: F1 toggles the minimap, F2 toggles shadows, F3 toggles decorative props and F4 shows renderer statistics.

Race and time trial behavior

In Race mode the player competes against one AI opponent for multiple laps. The AI uses the same selected vehicle type and is visible on both the track and minimap. In Time Trial mode the player races alone, and the game records lap times and local best laps for the selected vehicle and track. Checkpoints must be crossed in order; returning to the start/finish line after the intermediate checkpoints completes a lap.

9. Verification and Known Limitations

The project is verified with the production build command `bun run build`. Browser verification is currently manual: setup menu flow, scene startup, all three tracks, all three vehicles, race and time trial modes, countdown behavior, HUD and minimap updates, camera changes, headlights,

collisions, pause flow and finish screen. The renderer also exposes a runtime statistics panel for FPS.

Known limitations

- The physics model is intentionally arcade-style and does not use a rigid-body physics engine.
- The AI follows a centerline-based route and does not implement advanced overtaking strategy.
- The tracks are mostly planar so collision, minimap and AI logic remain reliable.

Conclusion

Kart Racing Simulator satisfies the core Interactive Graphics requirements with a complete browser-based racing scene, a complex team-authored hierarchical kart, multiple light and texture techniques, meaningful user interaction, and JavaScript-authored animations. Imported assets are used only as static models or media resources; the runtime behavior, animation systems and gameplay logic are implemented by the team in JavaScript.